



## **Application or use cases of various design patterns:**

### **Home Assignment 3: Implementing Resilience & AI Design Patterns**

#### **Group Participants:**

**Báez, Mauro Leandro**

**Manzur, Matías**

**Date: April 20th, 2025**

## General Description of our Application

The application is a prototype of a simple character creator, where you are able to create humanoid creatures but with animal body parts. Therefore, this app is of great use if the user needs a fast way of creating weird animal creatures for whatever reason they may need them (games, illustrations for stories, or just for fun).

This app offers a graphical user interface (GUI) where users can choose different body parts for the creature, combining them as they like. Also, users can undo or redo changes made to their fictional characters, as well as just generating a random combination of body parts if they are out of ideas.

In addition, users can now generate a brief story for their character using an LLM. They can choose between different models for this text generation process.

## Design Decisions and Patterns Applied

The main three advanced design patterns we decided to implement were: Proxy LLM, Timeout and Retry.

We hosted an Ollama server locally, which offers an API for text generation using different Large Language Models. Then, in our application, the OllamaClient class is in charge of the HTTP communication with the server.

In this communication is where we decided to implement both timeout and retry resilience patterns. The request is a simple HTTP POST and the client waits for a definite amount of time for the response. If this time exceeds the timeout limit, we assume there was an error in the server and an exception is raised.

Also, to implement the Retry pattern, any exception (including this timeout exception) is caught and the request is retried N times. If after N tries, the result is still not positive, then an error message is prompted to the user.

This Ollama client class is wrapped by the LLMProxy class. This class is responsible for caching the responses of the server given the same input arguments. This would be more useful in a real world scenario, where this would be used by multiple users, so the response for the same request made by different users can be returned much faster. We decided to cache responses for each model separately, to preserve the difference between each model's style and quality.

The user is able to choose between using TinyLlama and Llama 3.2 text generation models. We decided to use these models since they were the lightest models we could download and fit easily in a Docker image, while still being able to complete the task successfully. This could be extended to larger models, allowing the users to choose more clearly between a lower quality and fast response, or a high quality slow response.

We decided to construct the prompt for the different LLMs with the different options for body parts that the user selected. The used prompt was defined by trial and error until we observed that the quality of responses was better for both models.

## Insights

This assignment has given us an opportunity to use technologies which are on the rise, such as Ollama (run locally) with an emphasis on how to apply AI and resilience patterns. We now have a few more tools in our toolbox, which we will most probably use at some point in our career, especially if we are going to develop applications with AI features.

We have come to understand the value of resilience patterns such as Timeout and Retry, which are often overlooked because it's not always easy to consider what should happen when things don't go as expected in your software.

The AI pattern we chose to implement LLM Proxy prompted us to carefully consider what should be stored in the cache and for how long. This is definitely a question every developer should ask themselves. Otherwise, you risk falling into the trap of caching everything indiscriminately, which can lead to problems. In our toy example, this would be like serving cached results from one model when the user is actually trying to use another. These are all important considerations when designing any system but especially a system which uses LLMs.

## Possible Improvements

Many improvements come to mind:

- The ability of letting the user choose more models for the story generation.
- If the application included a wider variety of models, it could be customized to use different prompts depending on the model.
- Adding more options.
- Letting the user get a jpg or a pdf with the character and its story.
- In a real world application, the cache would need to expire after a certain amount of time.

All in all we feel like we accomplished the desired result of this assignment. We learned new things!